

ANTHROPIC 官方博客 • CLAUDE CODE

# 为每个任务 定制一套 Harness

Claude Code 动态工作流：让 Claude 现场编写  
专属的多智能体编排程序

---

2026 年 6 月 2 日 • Thariq Shihpar & Sid Bidasaria

# 什么是动态工作流？

面对复杂任务，Claude 不再在单个上下文窗口里硬扛，而是**现场编写一段编排脚本**——拆解任务、派生一组子智能体分工协作。

- 工作流是一个 **JavaScript 文件**，用特殊函数派生并协调子智能体
- 可为每个子智能体**指定模型**、决定是否在**隔离 worktree** 中运行
- 中途被打断？**可恢复续跑**，不必从头再来

你可以直接这样开口 ——

> 复现这个 flaky 测试，提出几个相互竞争的假设

> 挖掘我最近 50 个会话里反复出现的纠正，总结成规则

> 翻 6 个月的 Slack 事故记录，找出重复出现的根因

> 让多个智能体从不同视角批判我的商业计划

> 用工作流给 80 份简历排序，并安排候选人面试

# 引擎室：三个编排积木

## agent(prompt, opts?)

派生一个子智能体。**schema** 强制返回校验过的 JSON；**model** 选 opus/sonnet/haiku；**isolation** 隔离 worktree；**agentType** 用自定义子智能体

## parallel([fns])

扇出并发执行，**屏障语义**——等所有任务完成才返回

## pipeline(items, ...)

每个条目独立流过所有阶段，**无屏障**——快的不等慢的

**agent**(prompt, opts?): **Promise**<string | **JsonSchema**>

```
const bugs = await agent(  
  "audit auth.ts",  
  {  
    schema: BugList,  
    model: "haiku",  
    isolation: "worktree",  
    agentType: "reviewer",  
  })
```

----- **prompt** the agent's only input — **required**.

----- **schema** JSON Schema → validated JSON output.

----- **model** opus · sonnet · haiku. Omit = inherit.

----- **isolation** "worktree" (checkout) or "remote".

----- **agentType** custom / built-in subagent.

### RUN MANY — COMPOSE THE BLOCK

#### **parallel**([ fns ])

Fan out, run at once. Barrier — waits for all.

```
const all = await parallel(  
  files.map(f =>  
    () => agent(f)))
```

#### **pipeline**(items, ...)

Each item streams through every stage. No barrier.

```
await pipeline(items,  
  x => agent(draft(x)),  
  d => agent(check(d)))
```

图 1 • agent() 的完整签名，以及 parallel() 与 pipeline() 两种组合方式

# 为什么需要？单上下文的**三种失败模式**

## 智能体惰性

AGENTIC LAZINESS

多部分任务做到一半，Claude 就「觉得差不多了」，提前收工

## 自我偏好偏差

SELF-PREFERENTIAL BIAS

评判自己的产出时不自觉偏心，难以发现自身错误

## 目标漂移

GOAL DRIFT

轮次一多，对最初目标的忠实度逐渐流失，越跑越偏

工作流的解法：把工作拆给一组**目标单一、上下文干净**的子智能体，各自专注、互相制衡。

# 静态 vs 动态：量身定制的差距

**静态工作流**（Agent SDK / `claude -p`）：一次编写、通用执行，对各种输入都给出「泛化」的处理

**动态工作流**：Claude Opus 4.8 针对眼前这个任务现场编写专属 harness，深入你的代码和数据

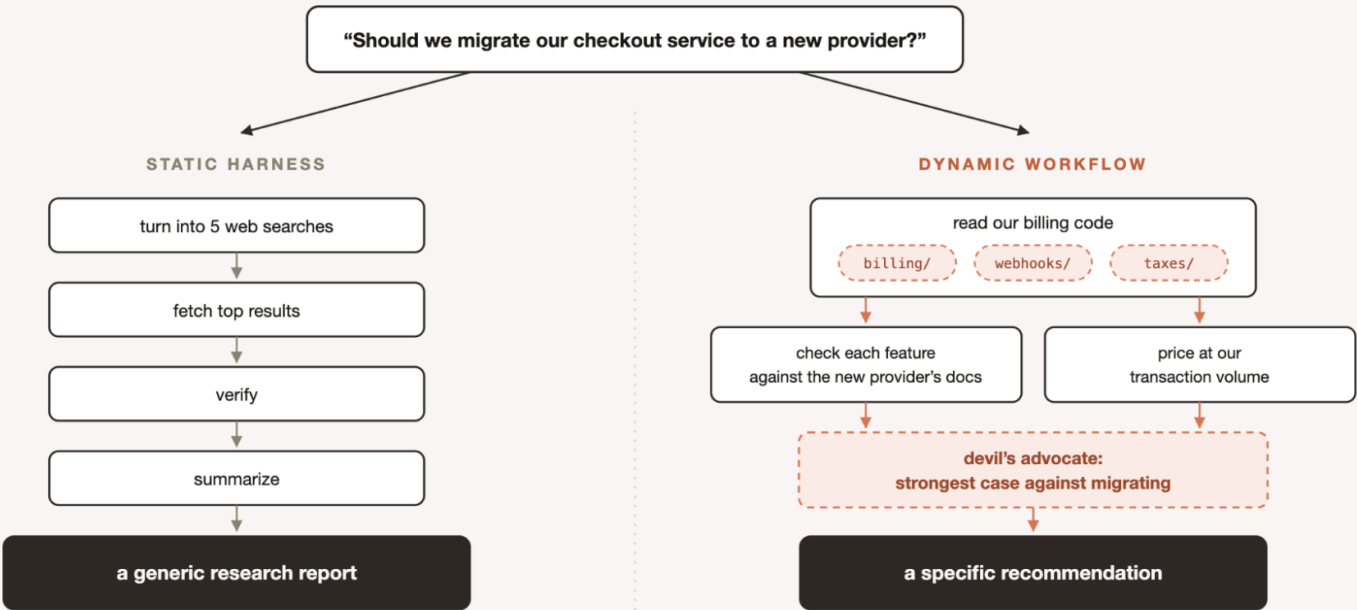


图 2 · 「该不该迁移结账服务？」—— 静态产出通用报告；动态会读你的账单代码、对照新供应商文档、再唱反调，给出针对性建议

PATTERNS

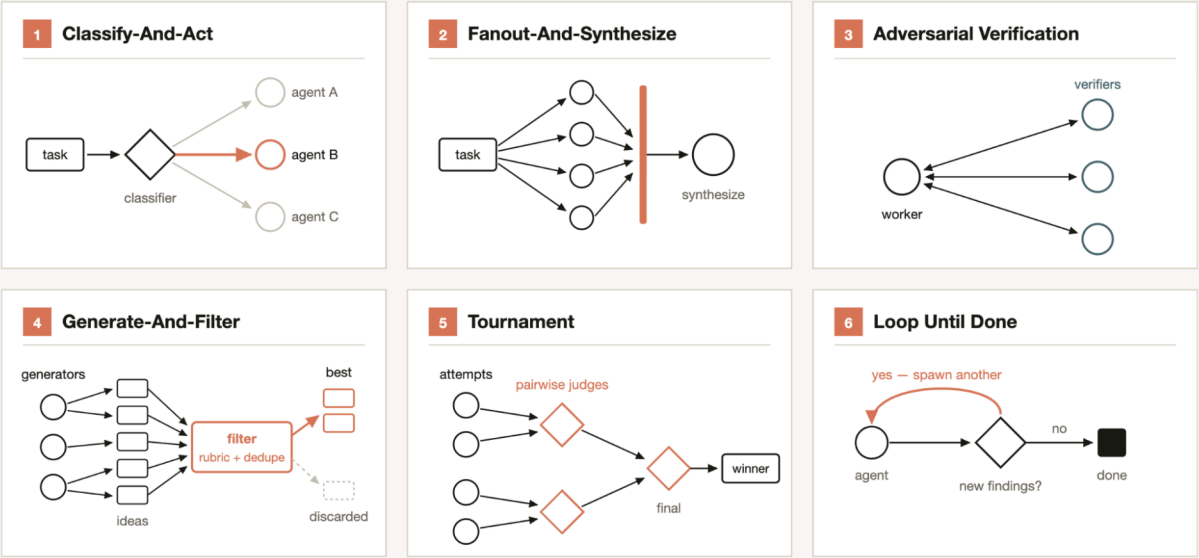
# 六种工作流模式

从分类路由到锦标赛对决 —— 可自由组合的编排原语

---

# 六种常用模式

## Six Workflow Patterns



- 1 · Classify-and-Act  
分类后行动 —— 按类型路由给不同智能体
- 2 · Fan-out-and-Synthesize  
扇出与综合 —— 拆小并行，再统一汇总
- 3 · Adversarial Verification  
对抗式验证 —— 独立智能体按标准校验产出
- 4 · Generate-and-Filter  
生成与过滤 —— 广撒网生成，按评分去重留优
- 5 · Tournament  
锦标赛 —— N 个方案两两对决，评委选出冠军
- 6 · Loop Until Done  
循环至完成 —— 跑到没有新发现为止

图 3 · 博客原图：六种 workflows 模式一览

USE CASES

# 十大应用场景

从大规模迁移到深度研究 —— workflows真正发力的地方

---



# 迁移重构 & 深度研究

## 迁移与重构

- 把迁移拆成具体步骤：调用点、失败测试、待改模块
- 每个修复派一个子智能体，跑在**隔离 worktree** 中
- **配对抗式审查智能体**把关，通过后再合并

★ 实战案例：Bun 从 Zig 改写为 Rust，用的就是这套方法

## 深度研究

- **扇出**网络搜索 → 抓取来源 → 对抗式核查每条断言
- 最终**综合**成一份带引用的研究报告
- 同样适用：Slack 状态汇总、代码库深度探索

💡 干净的上下文窗口，防止来源之间互相污染

# 深度验证：每条断言都有人核查

报告写完不等于可信。让 workflow 先提取每一条事实断言，再为每条断言派一个核查智能体对照来源验证，可选的「来源审计员」还会把关来源本身的质量。

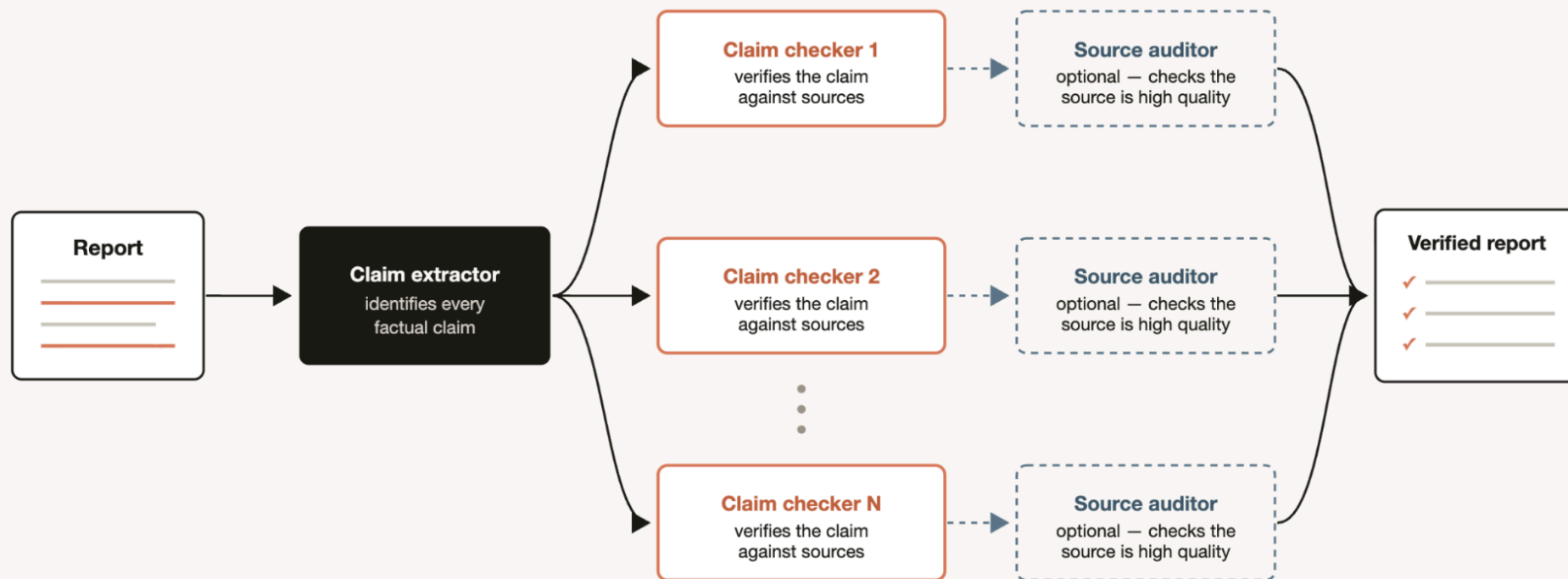


图 4 · 断言提取器 → N 个断言核查员 → 来源审计员 → 已验证报告

# 排序：千行数据的锦标赛

按「支持工单严重程度」这类定性标准给 1000+ 行数据排序？用锦标赛或流水线化的两两比较——每次对决都是一个全新智能体，不带任何历史包袱。

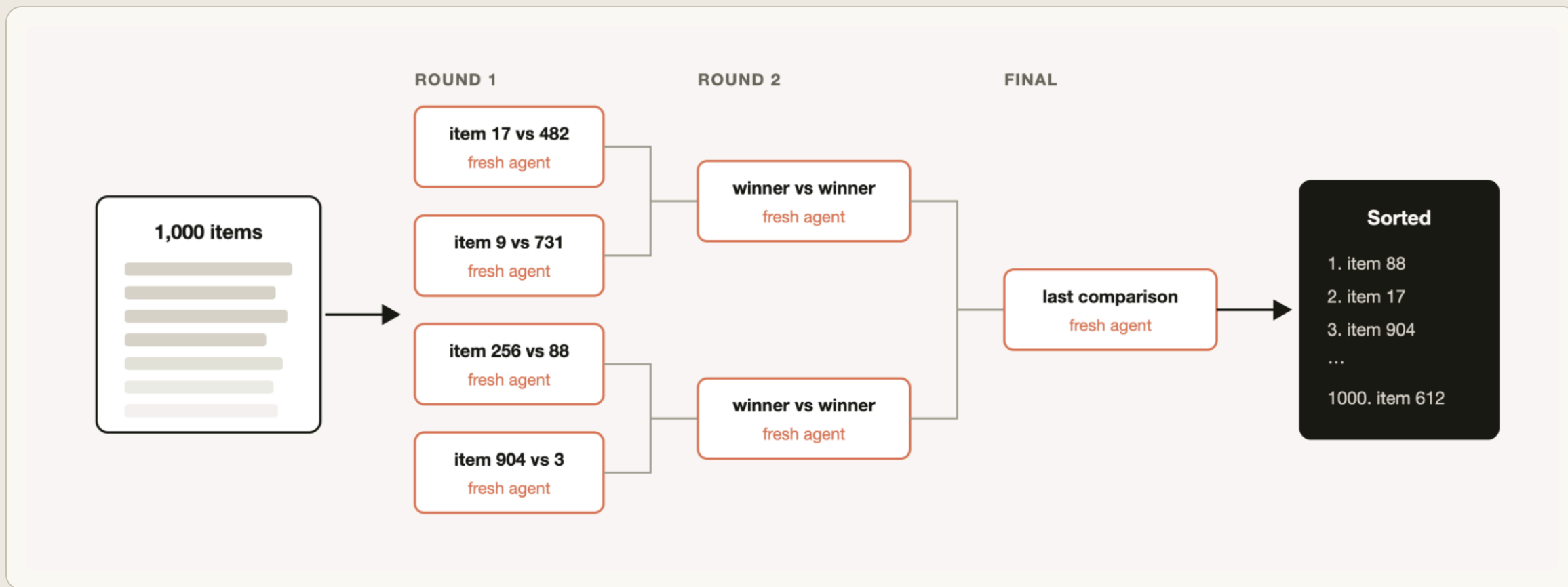


图 5 • 1000 个条目 → 多轮两两对决（每场都是 fresh agent）→ 完整排序结果

# 记忆与规则遵从：一条规则一个哨兵

CLAUDE.md 里的自定义规则总被忽略？为每条规则配一个专职验证智能体检查 diff，再由「怀疑论者」复核每个标记——是真违规，还是误报？

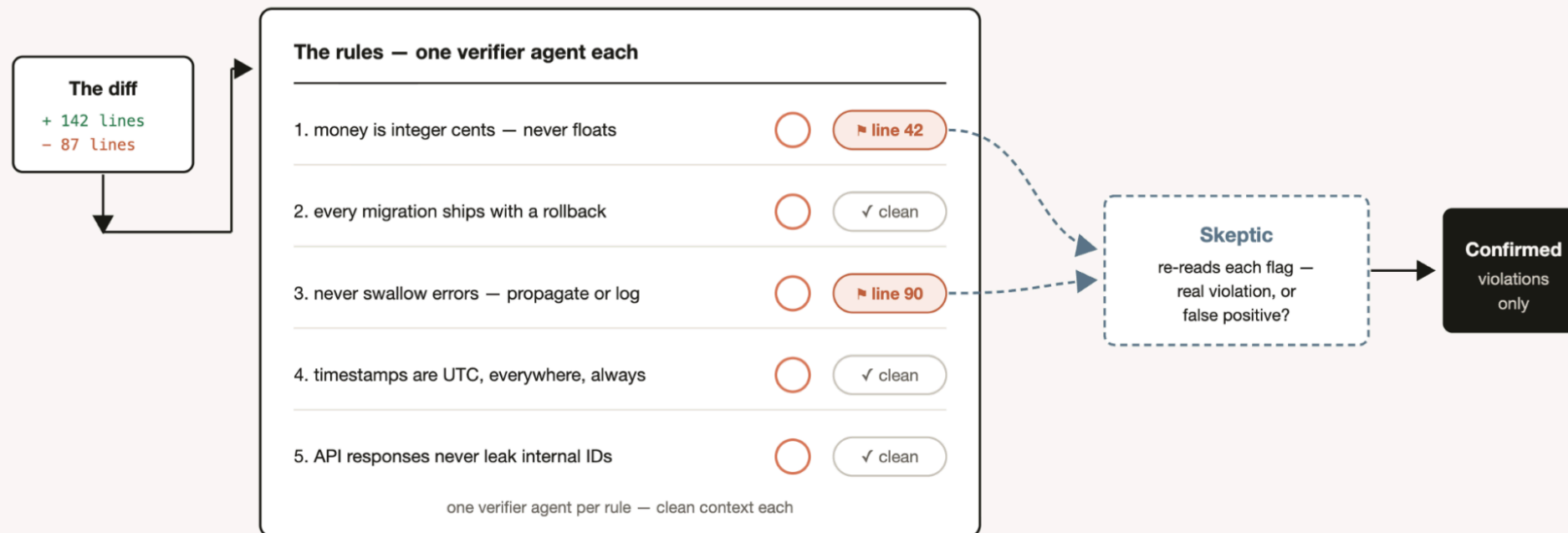


图 6 • 每条规则一个验证员（上下文各自干净）→ Skeptic 复核 → 只输出确认的违规

# 规模化分诊：隔离区设计

工单、bug 报告、用户反馈源源不断？分类 → 与已跟踪问题去重 → 修复或升级人工。关键安全设计：「隔离区」——读不可信内容的智能体**禁止**执行高权限操作，只向可信区传递结构化摘要。

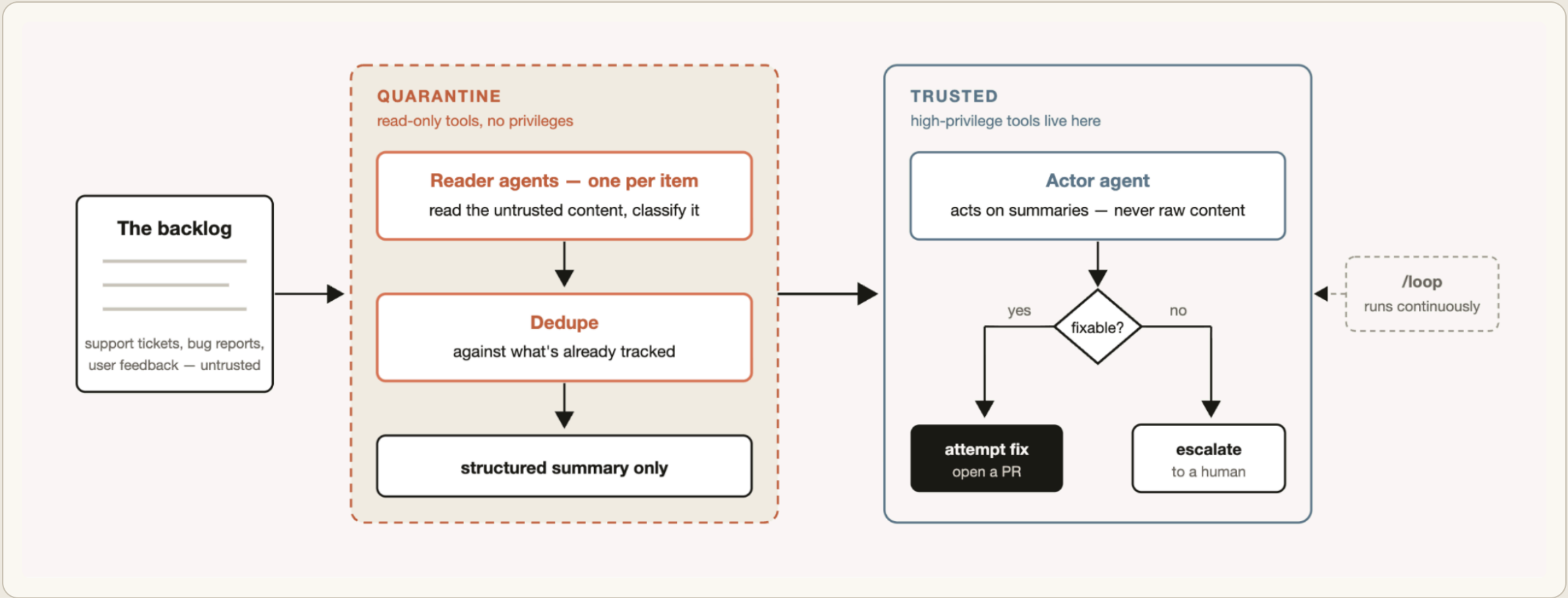


图 7 • 隔离区（只读、无特权）→ 结构化摘要 → 可信区行动智能体；配合 /loop 持续运行

# 更多场景：一招通吃

## 根因调查

多个智能体从**互不相交的证据**独立提出假设——调试、销售分析、数据工程、事后复盘都适用

## 探索与品味

设计或命名拿不定主意？按**评分标准**探索方案、配审查智能体，达标即停

## 轻量评测（Evals）

独立智能体各跑一遍任务，比较智能体按 **rubric** 打分——快速验证提示词或方案优劣

## 模型与智能路由

分类智能体先**预研任务复杂度**，再把工作路由给 Sonnet 或 Opus——好钢用在刀刃上

# 何时不用 & 实用技巧

## 👉 先想清楚再上

- 工作流的 **token** 消耗显著更高
- 留给**复杂、高价值**的任务
- 普通编码任务不需要三个 reviewer 围观

**提示要具体** —— 指名想要的模式效果最好；别忘了「快工作流」，小任务也能用

**搭配 `/loop` 与 `/goal`** —— 分诊、研究、验证这类可重复的工作流定期跑；用 `/goal` 设硬性完成标准

**设 **token** 预算** —— 一句「最多用 10k tokens」就能显式封顶开销

# 保存与分享：沉淀成团队资产

跑完后按 **s** 一键保存到 `~/.claude/workflows`

把 `.workflow.js` 放进 skill 文件夹、在 `SKILL.md` 中引用——装上这个 skill 的人，跑的就是同一个工作流

把保存的工作流当模板用，保留 Claude 现场调整的灵活性

```
Dynamic workflows
1 running · 2 completed

) ✓ review-changes · 14 agents · 482k tok · 6m 12s
  o find-flaky-tests · 6 agents · 121k tok · 1m 48s
  ✓ deep-research · 22 agents · 1.1M tok · 11m 3s

↑/↓ select · enter view · s save · esc close
```

Put the workflow file inside the skill folder

```
SKILL FOLDER

~/.claude/skills/deep-verify/

├─ SKILL.md
├─ verify-claims.workflow.js
└─ rubric.md
```

The workflow file ships inside the skill folder, right next to SKILL.md

```
SKILL.md

---

name: deep-verify
description: Verify every claim in a report

---

## Workflow

Run ./verify-claims.workflow.js to check each claim with its own subagent.
```

Share the folder — anyone who installs the skill runs the **same workflow**.

图 8 • `/workflows` 菜单按 `s` 保存；workflow 文件随 skill 文件夹一起分发



TAKEAWAY

# 让 Claude 自己搭脚手架

单上下文有惰性、偏心、漂移三种失败模式；

动态工作流用干净上下文 + 分工制衡来破解。

从复杂高价值任务开始，搭配 /loop、/goal 与 token 预算。

---

原文：[claude.com/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code](https://claude.com/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code)